Maze Solver Robot Project Report

Course: Embedded Systems (CSE234)
University: Alamein University - Faculty of Computer Science & Engineering

---

a) Input/Output Table

The following table describes the hardware interfaces used in the project, their roles, and their corresponding pins on the Arduino Mega 2560 (ATmega2560).

Component	Pin (Arduino)	Pin (AVR)	Direction	Description	Possible Values
Ultrasonic Front (Trig)	24	PA2	Output	Trigger signal for front sensor	High/Low
Ultrasonic Front (Echo)	25	PA3	Input	Echo signal for front sensor	Pulse width (Time)
Ultrasonic Right (Trig)	26	PA4	Output	Trigger signal for right sensor	High/Low
Ultrasonic Right (Echo)	27	PA5	Input	Echo signal for right sensor	Pulse width (Time)
Ultrasonic Left (Trig)	28	PA6	Output	Trigger signal for left sensor	High/Low
Ultrasonic Left (Echo)	29	PA7	Input	Echo signal for left sensor	Pulse width (Time)
Motor Left (ENA)	10	PB4 (OC2A)	Output	PWM speed control for left motors	0 - 255
Motor Left (IN1)	9	PH6	Output	Direction control for left motors	High/Low
Motor Left (IN2)	8	PH5	Output	Direction control for left motors	High/Low
Motor Right (IN3)	7	PH4	Output	Direction control for right motors	High/Low
Motor Right (IN4)	6	PH3	Output	Direction control for right motors	High/Low
Motor Right (ENB)	5	PE3 (OC3A)	Output	PWM speed control for right motors	0 - 255
Start Button (ON)	30	PC7	Input	Starts the maze-solving algorithm	High (Idle) / Low (Pressed)
Stop Button (OFF)	31	PC6	Input	Stops the robot immediately	High (Idle) / Low (Pressed)

b) Autonomous Robot Definition, Requirements, and Specifications

Definition

An autonomous maze-solving robot is an embedded system designed to navigate through a maze from a starting point to an exit without any human intervention. It uses sensors to perceive its environment and a microcontroller to execute a decision-making algorithm.

Requirements

- **Navigation:** Must be able to detect walls and openings in three directions (Front, Left, Right).
- **Autonomy:** Must start with a single switch and operate independently until the exit is reached or stopped.
- **Logic:** Must implement a rule-based algorithm (e.g., Left-Hand Rule) to ensure it eventually finds the exit.
- **Speed Control:** Must maintain a consistent speed while being able to perform precise turns.

Specifications

- **Controller:** ATmega2560 (Arduino Mega).
- **Sensors:** 3x HC-SR04 Ultrasonic Sensors.
- **Actuators:** 4x DC Motors with L298N Motor Driver.
- **Power Source:** 12V Battery Pack (for motors) and 5V (for logic).
- **Chassis:** 4-Wheel Drive robotic platform.

---

c) List of Sensors

Sensor Type	Model	Function
Ultrasonic Sensor	HC-SR04	Measures distance to walls by emitting sound waves and measuring the time for the echo to return.
Push Button	Standard Tactile	Acts as a user interface for starting and resetting the robot's state.

---

d) List of Peripherals

- **GPIO (General Purpose Input/Output):** Used for triggering ultrasonic sensors, reading button states, and controlling motor direction pins.
- **PWM (Pulse Width Modulation):** Used on Timer 2 and Timer 3 to control the speed of the DC motors via the L298N driver.
- **Timers:** Used for generating PWM signals and creating precise delays for sensor readings.

---

e) State Machine and Flow-Graph

State Machine

The robot operates in three primary states:

1. **STOPPED:** Idle state, waiting for the ON button.
2. **EXPLORING:** Actively moving and applying the Left-Hand Rule.
3. **FINISHED:** Reached the end of the maze (or stopped via OFF button).

Flow-Graph (Logic)

1. Start -> Wait for ON button.
2. Read Left, Front, and Right distances.

3. Is Left clear?
    - Yes -> Turn Left, Move Forward.
    - No -> Is Front clear?
        - Yes -> Move Forward.
        - No -> Is Right clear?
            - Yes -> Turn Right, Move Forward.
            - No -> Turn 180° (Dead End).
4. Repeat step 2 unless OFF button is pressed.

---

f) Main Function Sequence

The `main` function follows this sequence:

1. **Initialization:** Call `init_gpio()` and `init_pwm()` to configure registers.
2. **Polling Loop:** Continuously check the status of the ON and OFF buttons.
3. **State Update:** If ON is pressed, set `running` flag to 1. If OFF is pressed, set to 0 and stop motors.
4. **Execution:** If `running` is 1, call `get_distance()` for all sensors and execute the `if-else` logic for movement.

---

g) Software Design

Flowchart

The logic follows the **Left-Hand Rule**:

- **Priority 1:** If you can turn left, do it.
- **Priority 2:** If you can’t turn left, go straight.
- **Priority 3:** If you can’t go straight, turn right.
- **Priority 4:** If all else fails, turn around.

Required Functions

- `init_gpio()` : Configures DDRx and PORTx registers.
- `init_pwm()` : Configures TCCRx registers for motor speed control.
- `get_distance()` : Manages the trigger/echo timing to return distance in cm.
- `set_motors(speed_L, dir_L, speed_R, dir_R)` : Low-level control of H-Bridge pins.

---

h) AVR C Code

The code has been developed in pure C for the ATmega2560, avoiding Arduino-specific libraries to meet the project's technical requirements. The full code is provided in the `AVR_C_Code/main.c` file.

---

j) Test Strategy

Test Case	Scenario	Expected Behavior
Start/Stop	Press ON, then OFF.	Robot starts moving, then stops immediately.
Straight Path	Wall on both sides, clear front.	Robot moves forward in a straight line.
Left Opening	Opening appears on the left.	Robot turns 90° left and continues.
Dead End	Walls on Left, Front, and Right.	Robot performs a 180° turn.
Right Turn	Walls on Left and Front, clear Right.	Robot turns 90° right.
Sensor Accuracy	Object placed at 5cm.	Robot identifies it as a wall (Threshold=10cm).

Prepared by: Project Team
Date: May 12, 2026